Programación en Kotlin

PROGRAMACIÓN MULTIMEDIA Y DE DISPOSITIVOS MÓVILES

2º DAM

Índice

1. Introducción
2. Variables
3. Estructuras de control
4. Funciones
5. Clases
6. Colecciones
7. Nulabilidad

1. Introducción

Kotlin es un lenguaje de programación tipado y orientado a objetos que ya es considerado por *Google* como el lenguaje oficial de *Android* junto con *Java*.

Es totalmente interoperable con Java y puede correr sobre la JVM o un intérprete de *JavaScript*.

Android Studio soporta *Kotlin* desde su versión 3.0.

Página oficial: https://kotlinlang.org/

2. Variables

Es necesario utilizar las palabras reservadas ***var*** o ***val***:

- ***var***: Si el valor de la variable puede cambiar (mutable).
- ***val***: Si el valor de la variable no puede cambiar (solo lectura).

Kotlin es un lenguaje tipado pero no es necesario indicar el tipo de dato si se puede deducir del contexto (inferencia de tipo).

```
var n: Int = 1
var s: String = "Hola"
var d: Double = 0.1
```

```
var n = 1
var s = "Hola"
var d = 0.1
```

```
val num = 0
num = 1
```

Val cannot be reassigned

Change to 'var' Alt+Mayús+Intro

Todos sus tipos de datos básicos: https://kotlinlang.org/docs/reference/basic-types.html

2. Variables

En ***String*** se puede utilizar el carácter **+** para concatenar cadenas pero también el carácter **$**.

```
println("La palabra " + palabra + " tiene " + palabra.length + " letras.")
```

```
println("La palabra $palabra tiene ${palabra.length} letras.")
```

3. Estructuras de control

Básicamente son las mismas que *Java* salvo las siguientes diferencias:

- Se pueden utilizar expresiones con los ***if***

```
if (n1 > n2) {
    max = n1
}
else {
    max = n2
}
```

```
max = if (n1 > n2) {
    n1
} else {
    n2
}
```

3. Estructuras de control

Básicamente son las mismas que *Java* salvo las siguientes diferencias:

- ***switch*** es reemplazado por ***when*** y permite el uso de expresiones. Si devolvemos un resultado es obligatorio poner un ***else*** o cubrir todos los casos posibles.

```
when(x) {
    1    -> print("uno")
    2, 3 -> print("2 o 3")
    else -> {
        println("error")
        error = true
    }
}
```

```
res = when {
    num1 == 1                          -> 1
    palabra.contains( other: "con")    -> 2
    n in 1 ≤ .. ≤ 10                   -> 3
    num1 is Int                        -> 4
    else                               -> 5
}
```

3. Estructuras de control

Básicamente son las mismas que *Java* salvo las siguientes diferencias:

- El bucle ***for*** se asemeja al ***foreach***.

```
for (i in 1 ≤ .. ≤ 10) {
    println(i);
}
```

```
for (i in 3 ≥ downTo ≥ 1)
```



i irá tomando los valores 3 2 1

```
for (i in 1 ≤ .. ≤ 7 step 2)
```



i irá tomando los valores 1 3 5 7

```
for (c in "kotlin")
```



c irá tomando los valores ‘k’ ‘o’ ‘t’ ‘l’ ’i’ ‘n’

4. Funciones

Los métodos de *Java* en *Kotlin* se denominan funciones y se declaran con la palabra ***fun***.

El tipo ***void*** de Java en *Kotlin* es ***Unit***.

```
fun doble(n: Int): Int {
    return n * 2
}
```

```
fun doble(n: Int) = n * 2
```

Al realizar la llamada a la función podemos realizarla especificando los nombres de los parámetros de entrada. Si se realiza así puedes cambiar su orden.

```
fun suma(n1: Int, n2: Int): Int {
    return n1 + n2
}
```



```
suma( n1: 3, n2: 4)
```

```
suma(n2 = 4, n1 = 3)
```

4. Funciones

Además, podemos tener parámetros predeterminados, de tal forma que si no lo especificamos en la llamada, éste cogerá el valor por defecto.

```
fun suma(n1: Int = 1, n2: Int): Int {
    return n1 + n2
}
```



```
suma( n1: 3, n2: 4)
```

```
suma(n2 = 4)
```

4. Funciones

También podemos almacenar funciones dentro de variables. Para ello podemos utilizar dos métodos:

- Utilizar el operador de referencia de función **::**
- Utilizar una función lambda

```
fun main() {
    val saludo = ::hola
    saludo()
}

fun hola() {
    println("Hola!")
}
```

```
fun main() {
    val saludo = hola
    saludo()
}

val hola = {
    println("Hola!")
}
```

Kotlin sabe interpretar automáticamente que el valor de la variable *saludo* es una función, con lo que podemos llamarla como tal.

4. Funciones

Kotlin es capaz de identificar que la función *hola* no tiene parámetros y no devuelve ningún valor, pero hay veces que nos interesa especificarlo. La forma sería:

```
val hola: () -> Unit = {
    println("Hola!")
}
```

4. Funciones

En las funciones lambda podemos omitir el parámetro utilizando la palabra reservada ***it***.

```
val precio: (Int) -> String = { cantidad ->
    "$cantidad euros"
}
```

```
val precio: (Int) -> String = {
    "$it euros"
}
```

5. Clases

Por defecto, todas las clases son públicas y heredan de la clase ***Any*** (similar al ***Object*** de *Java*).

```
class Persona (val nombre: String, val edad: Int) {
    var departamento: String = ""
}
```

5. Clases

Las clases tienen 2 tipos de **constructores**, el **principal** y el **secundario**.

Una clase solo puede tener un **constructor principal**, que se define como parte del encabezado de la clase. Puede ser predeterminado o con parámetros y no puede contener código.

```
class Persona (val nombre: String, val edad: Int) {
    var departamento: String = ""
}
```



Constructor principal con parámetros

```
class Persona {
    var nombre: String = ""
    var edad: Int = 0
    var departamento: String = ""
}
```



Constructor principal sin parámetros

5. Clases

Las clases tienen 2 tipos de **constructores**, el **principal** y el **secundario**.

Una clase puede tener varios **constructores secundarios**. Puedes definir ser con o sin parámetros y tiene un cuerpo con código. Si la clase tiene un constructor principal, cada constructor secundario debe inicializarlo.

```
class Persona (val nombre: String, val edad: Int, val departamento: String) {

    constructor(nombre: String, edad: Int) : this(nombre, edad,  departamento: ""){

    }
}
```

5. Clases

Cuando no defines las funciones ***get*** y ***set*** para un atributo, el compilador de *Kotlin* crea las funciones a nivel interno. No las veremos en nuestro código.

Las variables de tipo ***val*** son variables de solo lectura, por lo que no tienen funciones ***set***.

Por ejemplo, para la variable *num1* crea internamente las siguientes funciones:

```
var num1 = 1
    get() = field
    set(value) {
        field = value
    }
```

5. Clases

El identificador ***field*** hace referencia al campo de respaldo del atributo, que básicamente es una variable interna que el compilador de *Kotlin* genera automáticamente y que se guarda en memoria.

Se necesita para leer el valor del atributo en la función ***get*** o para actualizar su valor con la función ***set***.

El parámetro ***value*** es el encargado de recoger el valor a asignar al atributo.

```
var num1 = 1
    get() = field
    set(value) {
        field = value
    }
```

```
var num1 = 1
```

5. Clases

Podemos modificar el método ***set*** para, por ejemplo, asegurarnos de que al asignar un valor cumpla unas condiciones. Si no las cumple cambiará su valor.

```
var num1 = 1
    set(value) {
        if (value in 1 ≤ .. ≤ 10) {
            field = value
        }
    }
```

5. Clases: Herencia

La herencia se realiza mediante los **:**

Para indicar que una clase puede ser heradada se le debe añadir la palabra reservada ***open***, así como a los métodos que pueden ser sobreescritos. Estos se indicarán con la palabra reservada ***override***.

```
open class Persona(
    val nombre: String,
    val apellido: String,
    val edad: Int
) {
    open fun imprimirInformacion() {
        println("Nombre: $nombre $apellido, Edad: $edad")
    }
}

class Alumno(
    nombre: String,
    apellido: String,
    edad: Int,
    val grupo: Char
) : Persona(nombre, apellido, edad) {
    override fun imprimirInformacion() {
        super.imprimirInformacion()
        println("Grupo: $grupo")
    }
}
```

5. Clases: Interfaces

La implementación de una interfaz también se realiza mediante los **:**

Si hay más de una clase a heredar/implementar se separa entre comas. Al igual que en *Java*, *Kotlin* no permite la herencia múltiple pero sí implementar varias interfaces.

```
interface Interfaz {

}

class Profesor : Interfaz {

}
```

5. Clases: Modificadores

Los modificadores de visibilidad en *Kotlin* son ***public***, ***private***, ***protected*** e ***internal***.

internal permite que se pueda acceder a la declaración en el mismo módulo es similar al privado, pero puedes acceder a propiedades y métodos internos desde fuera de la clase, siempre y cuando lo hagas en el mismo módulo.

Modificador	Accesible en la misma clase	Accesible en subclase	Accesible en el mismo módulo	Accesible fuera del módulo
private	✓	X	X	X
protected	✓	✓	X	X
internal	✓	✓	✓	X
public	✓	✓	✓	✓

5. Clases: Delegación

Por último, *Kotlin* ofrece una técnica de delegación de propiedades.

Para eso utilizamos el operador ***by*** de tal forma que delegaremos la implementación de una interfaz o clase a otro objeto. Esto permite que una clase derive funcionalidad de otra clase sin herencia directa.

```
var nombre by remember { mutableStateOf( value: "") }
```

5. Clases: Creación de objetos

Para la creación del objeto no se utiliza la palabra **new**, ni indicar el tipo del objeto, simplemente se deduce del constructor.

```
val persona = Persona( nombre: "Pepe",  edad: 30)
```

Cuando definimos la variable con la palabra clave ***val*** para hacer referencia al objeto, la variable en sí es de solo lectura, pero el objeto de la clase permanece mutable. Esto significa que no puedes reasignar otro objeto a la variable, pero puedes cambiar el estado del objeto cuando actualices los valores de sus propiedades.

5. Clases: Generalidad

La generalidad en *Kotlin* es similar a la de *Java*.

```
class Pregunta<T>(
    val enunciado: String,
    val respuesta: T,
    val dificultad: String
)

fun main() {
    val pregunta1 = Pregunta<String>( enunciado: "¿Cuál es la capital de España?", respuesta: "Madrid", dificultad: "fácil")
    val pregunta2 = Pregunta<Boolean>( enunciado: "Verdadero o falso. La caja negra de un avión es negra.", respuesta: false, dificultad: "media")
}
```

5. Clases: Clases de enumeración

Una clase ***enum*** se usa para crear tipos con un conjunto limitado de valores posibles.

```
enum class Dificultad {
    FACIL, MEDIO, DIFICIL
}

class Pregunta<T>(
    val enunciado: String,
    val respuesta: T,
    val dificultad: Dificultad
)

fun main() {
    val pregunta1 = Pregunta<String>( enunciado: "¿Cuál es la capital de España?",  respuesta: "Madrid", Dificultad.FACIL)
    val pregunta2 = Pregunta<Boolean>( enunciado: "Verdadero o falso. La caja negra de un avión es negra.",  respuesta: false, Dificultad.MEDIO)
}
```

5. Clases: Clases de datos

Una clase ***data*** es aquella que solo tiene datos y no tiene funciones que realicen acciones.

En nuestro ejemplo la clase *Pregunta* podría ser una clase ***data***.

```
enum class Dificultad {
    FACIL, MEDIO, DIFICIL
}

data class Pregunta<T>(
    val enunciado: String,
    val respuesta: T,
    val dificultad: Dificultad
)

fun main() {
    val pregunta1 = Pregunta<String>( enunciado: "¿Cuál es la capital de España?",  respuesta: "Madrid", Dificultad.FACIL)
    val pregunta2 = Pregunta<Boolean>( enunciado: "Verdadero o falso. La caja negra de un avión es negra.",  respuesta: false, Dificultad.MEDIO)
}
```

5. Clases: Objetos Singleton

Un *singleton* es una clase que solo puede tener una única instancia. *Kotlin* proporciona una construcción especial, llamada ***object***, que se puede usar para crear una clase *singleton*.

```
object Progreso {
    var falladas: Int = 0
    var acertadas: Int = 0
}
```

```
Progreso.falladas++
Progreso.acertadas++
```

También podemos definir objetos *singleton* dentro de clases y llamar a sus atributos desde dentro de la clase. Esto es similar al ***static*** de *Java*.

```
companion object Progreso {
    var falladas: Int = 0
    var acertadas: Int = 0
}
```

```
falladas++
acertadas++
```

6. Colecciones: Arrays

Para declarar un *array* se utiliza la función ***arrayOf***. Para acceder a los elementos se utiliza los operadores **[]**.

```
val dias = arrayOf<String>("Lunes", "Martes", "Miércoles", "Jueves")
```

6. Colecciones: Listas

Dos de los tipos de colección más utilizados en *Kotlin* son ***List*** y ***MutableList***.

- ***List*** es una interfaz que define las propiedades y las funciones relacionadas con una colección ordenada de solo lectura de los elementos.
- ***MutableList*** extiende la interfaz ***List*** con la definición de funciones para modificar una lista, como agregar o quitar elementos.

```
val dias = listOf("Lunes", "Martes", "Miércoles", "Jueves")
```

```
val dias = mutableListOf("Lunes", "Martes", "Miércoles", "Jueves")
```

6. Colecciones: Listas

Tanto en ***List*** como en ***MutableList*** podremos utilizar ***[]*** y la función ***get*** para acceder a sus elementos, la propiedad ***size*** para obtener su tamaño o la función ***indexOf*** para obtener el índice dado un elemento.

Para recorrer ambas listas podremos utilizar el bucle ***for***:

```
for(dia in dias){
    println(dia)
}
```

En el caso de ***MutableList*** además podremos utilizar la función ***add***, ***remove*** o asignar un nuevo valor a una posición en concreto accediendo a ella con ***[]***.

6. Colecciones: Conjuntos

Un ***conjunto*** es una colección que no tiene un orden específico ni permite valores duplicados, que utiliza códigos *hash* como índices.

Para crear un conjunto en Kotlin se utiliza las clases ***Set*** y ***MutableSet***.

```
val dias = mutableSetOf("Lunes", "Martes", "Miércoles", "Jueves")
```

Como los conjuntos son una colección sin ordenar no podremos acceder a un elemento según su índice ya que los conjuntos no tienen índices. Podremos comprobar si un elemento está en el conjunto con las funciones ***contains*** o ***in***.

```
if(dias.contains("Lunes")){

}
```

```
if("lunes" in dias){

}
```

6. Colecciones: Conjuntos

Podremos utilizar la propiedad ***size*** para obtener el tamaño del conjunto y las funciones ***add*** o ***remove*** para añadir o quitar elementos en el conjunto.

6. Colecciones: Mapas

Un *mapa* es una colección que consta de claves y valores donde las claves, que serán únicas, se asignan a valores, que no tienen por qué ser únicos.

Haremos uso de las clases ***Map*** y ***MutableMap***.

```
val notas = mutableMapOf(
    "AD" to 7,
    "DI" to 5,
    "PMDM" to 8,
    "PSP" to 7,
    "SGE" to 9
)
```

Podremos utilizar los ***[]*** (indicando la clave dentro) o la función ***get*** para obtener los valores, ***remove*** para eliminar o asignar un valor accediendo con ***[]*** o con la función ***put***.

7. Nulabilidad

Kotlin distingue entre tipos anulables y no anulables:

- Tipos anulables son los que pueden se *null*.
- Tipos no anulables son los que no pueden ser *null*.

Para declarar tipos anulables se debe de añadir el operador **?** Al final del tipo.

```
var num: Int? = 1
```

Si quiero hacer uso de una función o propiedad anulable debo utilizar el operador de llamada segura **?.**

```
var nombre: String? = "Pepe"
println(nombre.length)
```

Only safe (?.) or non-null asserted (!!.) calls are allowed on a nullable receiver of type String?

Surround with null check Alt+Mayús+Intro More actions... Alt+Intro

```
var nombre: String? = "Pepe"
println(nombre?.length)
```

7. Nulabilidad

También podemos utilizar el operador de aserción no nulo !! para indicar que el valor de la variable o función es no nulo (aunque no lo sea).

Hay que tener mucho cuidado porque puede producir una excepción del tipo **NullPointerException**.

```
var nombre: String? = "Pepe"
println(nombre!!.length)
```